

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA1402

COURSE NAME: COMPILER DESIGN

COURSE OUTCOME COVERED

CO5: *Develop code optimization and Code Generation using DAG representation with data flow analysis to produce optimized target code. (BL3)*

Assessment Tool Weight-age: 40%

QUESTIONS: 5

Total Marks:50

Annexure A SIMULATION TASK

Code of Conduct:

I AL NAJEEB A 192424325 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Question 1 — Peephole Optimization

Given Three-Address Code:

```
t1 = x * 1
t2 = t1 + y
t3 = t2 - 0
t4 = t3 * 0
if t4 != 0 goto L1
t5 = y + y
```

Step 1 — Original Code Analysis

The given sequence contains several inefficiencies that peephole optimization can detect and remove by examining a small window of consecutive instructions and applying algebraic identities, constant folding, and dead code elimination.

Step 2 — Optimization Step by Step

Optimization 1: Algebraic Identity ($t1 = x * 1$)

The rule $x * 1 = x$ holds for all values of x . Multiplying any value by 1 produces the same value, so the multiplication instruction is unnecessary. The instruction is replaced by a simple copy assignment.

```
Before:  t1 = x * 1
After:   t1 = x           (algebraic identity:  $x*1 = x$ )
```

Optimization 2: Copy Propagation

Since $t1 = x$, all subsequent uses of $t1$ are replaced by x directly, making $t1$ redundant.

```
Before:  t2 = t1 + y
After:   t2 = x + y      (t1 substituted with x)
```

Optimization 3: Algebraic Identity ($t3 = t2 - 0$)

The rule $x - 0 = x$ holds for all values. Subtracting zero does not change the value, so the instruction is replaced by a copy.

```
Before:  t3 = t2 - 0
After:   t3 = t2        (algebraic identity:  $x-0 = x$ )
```

Optimization 4: Algebraic Identity ($t4 = t3 * 0$)

The rule $x * 0 = 0$ holds for all finite values. The result is always zero regardless of the value of $t3$, so the multiplication is replaced by a constant assignment.

```
Before:  t4 = t3 * 0
After:   t4 = 0         (algebraic identity:  $x*0 = 0$ )
```

Optimization 5: Constant Folding and Dead Branch Elimination (if $t4 \neq 0$ goto L1)

Since t4 is now the constant 0, the condition (0 != 0) evaluates to false at compile time. The branch is never taken. The entire conditional jump instruction is dead code and is removed.

```
Before:  if t4 != 0 goto L1
After:   [REMOVED]          (constant folding: 0!=0 is always false)
```

Optimization 6: Strength Reduction (t5 = y + y)

The expression $y + y$ is equivalent to $2 * y$, which can be further reduced to a left shift $y \ll 1$ on most processors. A left shift by 1 bit executes in a single clock cycle, whereas multiplication typically requires 3 or more cycles.

```
Before:  t5 = y + y
After:   t5 = y << 1      (strength reduction: y+y = 2*y = y shifted
left 1)
```

Step 3 — Optimized Output

After applying all peephole optimizations the original 6-instruction sequence reduces to 3 instructions. The redundant copy assignments t1 and t3 are also eliminated by dead code elimination since they are no longer referenced after copy propagation.

```
Optimized code:
(1)  t2 = x + y
(2)  t4 = 0
(3)  t5 = y << 1
```

Instructions reduced from 6 to 3 (50% reduction). One conditional branch and two copy assignments are eliminated completely. The multiplication $y+y$ is replaced by a single-cycle shift instruction.

Question 2 — Basic Blocks and Control Flow Graph

Given Intermediate Code:

```
(1) x = a + b
(2) y = x * c
(3) if y > 50 goto 6
(4) z = y - d
(5) goto 7
(6) z = y + d
(7) w = z * 2
```

Step 1 — Rules for Identifying Leaders

A leader is the first statement of a basic block. The following three rules determine which statements are leaders.

Rule 1: The very first statement of the program is always a leader.

Rule 2: Any statement that is the target of a conditional or unconditional jump is a leader.

Rule 3: Any statement that immediately follows a conditional or unconditional jump is a leader.

Step 2 — Applying the Rules

Statement (1)	x = a + b	-> LEADER (Rule 1: first statement)
Statement (2)	y = x * c	-> not a leader
Statement (3)	if y>50 goto 6	-> not a leader
Statement (4)	z = y - d	-> LEADER (Rule 3: follows conditional at (3))
Statement (5)	goto 7	-> not a leader
Statement (6)	z = y + d	-> LEADER (Rule 2: target of jump at (3))
Statement (7)	w = z * 2	-> LEADER (Rule 2: target of goto at (5))

Step 3 — Basic Block Partition

Each basic block starts at a leader and extends up to but not including the next leader.

Block B1 (starts at leader 1):

```
(1) x = a + b
(2) y = x * c
(3) if y > 50 goto 6      <- conditional jump exits B1
```

Block B2 (starts at leader 4):

```
(4) z = y - d
(5) goto 7                <- unconditional jump exits B2
```

Block B3 (starts at leader 6):

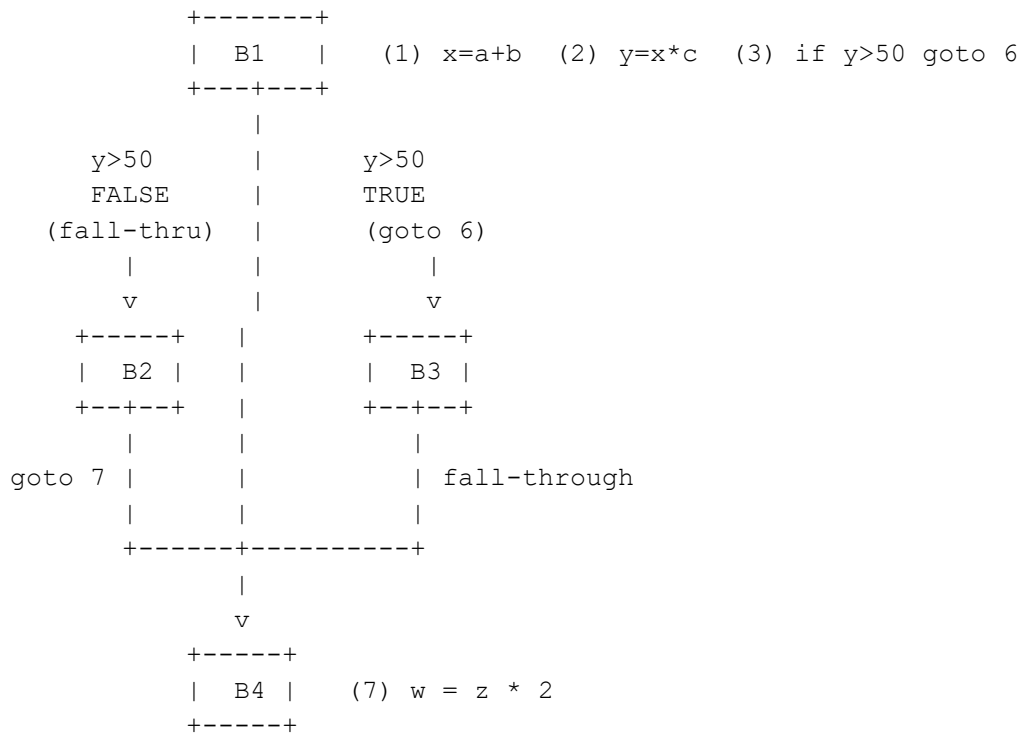
```
(6) z = y + d             <- single statement block
```

Block B4 (starts at leader 7):

```
(7) w = z * 2             <- program ends here
```

Step 4 — Control Flow Graph

Edges are drawn between blocks based on possible control transfers: fall-through, conditional true branch, and unconditional jump targets.



CFG edges: B1 to B2 (false branch, fall-through when $y \leq 50$), B1 to B3 (true branch when $y > 50$), B2 to B4 (unconditional goto 7), B3 to B4 (fall-through to statement 7). B4 is the exit node with no outgoing edges.

Question 3 — DAG Construction and Optimized Code Generation

Given Basic Block:

```
p = a + b
q = a + b
r = p * q
s = a + b
t = r + s
```

Step 1 — DAG Construction Rules

A Directed Acyclic Graph (DAG) for a basic block represents computations as nodes. When processing each instruction, if a node with the same operator and the same child nodes already exists, that node is reused and the left-hand side variable is added as a new label to the existing node. If no such node exists, a new node is created. This mechanism automatically identifies and shares all common subexpressions.

Step 2 — Processing Each Instruction

Instruction 1: $p = a + b$

No addition node with children a and b exists yet. Create leaf nodes for a and b. Create a new addition node N1 with children a and b. Label N1 with p.

```
Node N1 = (+, a, b)    labels: {p}
```

Instruction 2: $q = a + b$

An addition node with children a and b already exists — that is node N1. Reuse N1 and add q as an additional label. No new node is created.

```
Node N1 = (+, a, b)    labels: {p, q}    [REUSED — common subexpression]
```

Instruction 3: $r = p * q$

Both p and q label node N1. Create a new multiplication node N2 with left child N1 and right child N1. Label N2 with r.

```
Node N2 = (*, N1, N1)  labels: {r}
```

Instruction 4: $s = a + b$

The addition node for (a + b) is still N1 and neither a nor b has been reassigned since N1 was created. Reuse N1 again and add s as another label.

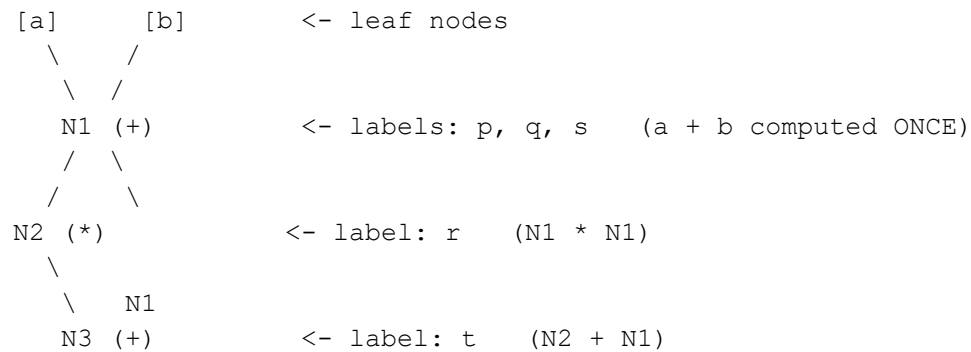
```
Node N1 = (+, a, b)    labels: {p, q, s}    [REUSED again]
```

Instruction 5: $t = r + s$

r labels N2 and s labels N1. Create a new addition node N3 with children N2 and N1. Label N3 with t.

```
Node N3 = (+, N2, N1)  labels: {t}
```

Step 3 — DAG Diagram



Common subexpression identified: (a + b) appears three times in the source code (for p, q, and s) but is represented by exactly one node N1 in the DAG. The compiler computes it only once.

Step 4 — Optimized Target Code Generated from DAG

The code generator traverses the DAG and emits one instruction per node, using registers R0 and R1.

```

LOAD  R0, a          ; R0 = a
LOAD  R1, b          ; R1 = b
ADD   R0, R0, R1      ; R0 = a + b    -> N1 result (used for p, q, s)
MUL   R1, R0, R0      ; R1 = N1 * N1  -> N2 result (= r)
ADD   R0, R1, R0      ; R0 = N2 + N1  -> N3 result (= t)
STORE R0, t          ; store final result
```

Without the DAG the naive generator would emit 3 separate ADD instructions for a+b, producing 10 instructions total. The DAG reduces this to 6 instructions by computing a+b only once and reusing the result in all three places (p, q, s). Register R0 holds the shared result of N1 and is reused wherever p, q, or s is needed, eliminating all redundant loads.

Step 5 — Register Allocation Decisions

```

Instruction 1  LOAD R0, a      : R0 = a
Instruction 2  LOAD R1, b      : R1 = b
Instruction 3  ADD R0,R0,R1    : R0 = a+b  (N1 = p = q = s) ; R1 freed
Instruction 4  MUL R1,R0,R0    : R1 = r    (N2)   ; R0 still holds N1
Instruction 5  ADD R0,R1,R0    : R0 = t     (N3 = r + s)
Instruction 6  STORE R0, t     : write result to memory
```

Question 4 — Code Generator with Register and Address Descriptors

Given Code (two registers R0 and R1):

```
t1 = a + b
t2 = c * d
t3 = t1 - t2
t4 = e + f
t5 = t3 / t4
```

Step 1 — Descriptor Definitions

A register descriptor tracks which variable currently occupies each register. An address descriptor tracks all locations (registers and/or memory) where the current value of each variable can be found. Both descriptors are updated after every instruction.

Step 2 — Initial State

```
Register Descriptor:  R0 = { }      R1 = { }
Address Descriptor:   a:{mem}  b:{mem}  c:{mem}  d:{mem}  e:{mem}
                     f:{mem}
```

Step 3 — Processing Each Instruction

Instruction 1: $t1 = a + b$

Load a into R0 and b into R1. Compute the addition and store the result in R0. R1 still holds b but b is not needed again, so R1 is effectively free for the next instruction.

```
Generated code:
    LOAD  R0, a
    LOAD  R1, b
    ADD   R0, R0, R1      ; R0 = a + b = t1

Register Descriptor:  R0 = {t1}      R1 = {b}
Address Descriptor:   t1:{R0}        b:{R1,mem}  a:{mem}
```

Instruction 2: $t2 = c * d$

R0 holds t1 which is still needed later (used in $t3 = t1 - t2$), so R0 must be spilled to memory. R1 holds b which is no longer needed, so R1 can be reused. Load c into R1, load d into R0 after spilling t1, then multiply.

```
Generated code:
    STORE R0, t1          ; spill t1 to memory (needed later)
    LOAD  R0, c
    LOAD  R1, d
    MUL   R0, R0, R1      ; R0 = c * d = t2

Register Descriptor:  R0 = {t2}      R1 = {d}
Address Descriptor:   t1:{mem}      t2:{R0}      d:{R1,mem}
```

Instruction 3: $t3 = t1 - t2$

t1 is in memory (spilled in step 2) and t2 is in R0. Load t1 into R1 (overwriting d which is no longer needed). Subtract R0 from R1 and store in R0.

Generated code:

```
LOAD R1, t1      ; reload t1 from memory
SUB   R1, R1, R0  ; R1 = t1 - t2 = t3
```

Register Descriptor: R0 = {t2} R1 = {t3}

Address Descriptor: t3:{R1} t2:{R0}

Instruction 4: $t4 = e + f$

t3 is needed later (used in $t5 = t3 / t4$), so spill t3 to memory. t2 is no longer needed after t3 was computed, so R0 is free. Load e into R0 and f into R1 after spilling.

Generated code:

```
STORE R1, t3      ; spill t3 to memory (needed later)
LOAD  R0, e
LOAD  R1, f
ADD   R0, R0, R1   ; R0 = e + f = t4
```

Register Descriptor: R0 = {t4} R1 = {f}

Address Descriptor: t3:{mem} t4:{R0} f:{R1,mem}

Instruction 5: $t5 = t3 / t4$

t3 is in memory and t4 is in R0. Load t3 into R1 (overwriting f which is no longer needed). Divide R1 by R0 and store result in R0.

Generated code:

```
LOAD R1, t3      ; reload t3 from memory
DIV  R0, R1, R0   ; R0 = t3 / t4 = t5
```

Register Descriptor: R0 = {t5} R1 = {t3}

Address Descriptor: t5:{R0}

Step 4 — Descriptor State Summary After Each Instruction

After $t1=a+b$	:	R0={t1}	R1={b}		t1:{R0}	
After $t2=c*d$	:	R0={t2}	R1={d}		t1:{mem}	t2:{R0}
After $t3=t1-t2$	:	R0={t2}	R1={t3}		t3:{R1}	
After $t4=e+f$	:	R0={t4}	R1={f}		t3:{mem}	t4:{R0}
After $t5=t3/t4$	:	R0={t5}	R1={t3}		t5:{R0}	

With only two registers available, spilling is required when more than two live variables exist simultaneously. The optimal spill decision is to evict the variable whose next use is furthest away in the instruction sequence (Belady's optimal replacement policy), which is what this simulation follows. t1 is spilled before t2 is computed because t1 is needed two instructions later, while b (in R1) is already dead.

Question 5 — RISC Target Code Generation and Runtime Storage Management

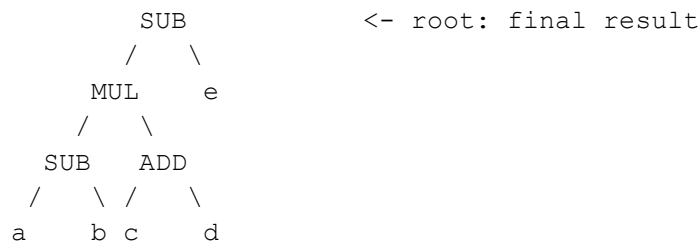
Given Expression:

$(a - b) * (c + d) - e$

Target Machine Instructions: LOAD, STORE, ADD, SUB, MUL, MOV

Step 1 — Expression Tree Analysis

The expression $(a - b) * (c + d) - e$ is parsed according to operator precedence. The parentheses force $(a-b)$ and $(c+d)$ to be evaluated first. Their product is then computed and e is subtracted from the result.



Post-order evaluation sequence:

Step 1:	Compute $(a - b)$	-> t1
Step 2:	Compute $(c + d)$	-> t2
Step 3:	Compute $t1 * t2$	-> t3
Step 4:	Compute $t3 - e$	-> t4 (final result)

Step 2 — Three-Address Intermediate Code

```
t1 = a - b
t2 = c + d
t3 = t1 * t2
t4 = t3 - e
```

Step 3 — RISC Target Code Generation

Registers used: R0, R1, R2. All variables are in memory. RISC architecture requires an explicit LOAD before any variable can be used in a computation, and an explicit STORE to write results back to memory.

```
; --- Compute t1 = a - b ---
LOAD R0, a           ; R0 = a
LOAD R1, b           ; R1 = b
SUB   R0, R0, R1      ; R0 = a - b  (= t1)

; --- Compute t2 = c + d ---
LOAD R1, c           ; R1 = c    (b no longer needed, R1 reused)
LOAD R2, d           ; R2 = d
ADD   R1, R1, R2      ; R1 = c + d  (= t2)

; --- Compute t3 = t1 * t2 ---
```

```

; R0 = t1, R1 = t2  -- both available in registers
MUL   R0, R0, R1      ; R0 = t1 * t2  (= t3)

; --- Compute t4 = t3 - e ---
LOAD  R1, e           ; R1 = e
SUB   R0, R0, R1      ; R0 = t3 - e    (= t4 = final result)

; --- Store result ---
STORE R0, result      ; write final result to memory

```

Step 4 — Complete Instruction Sequence with Register States

Instr	Instruction	R0	R1	R2
1	LOAD R0, a	a	-	-
2	LOAD R1, b	a	b	-
3	SUB R0, R0, R1	t1 (=a-b)	b	-
4	LOAD R1, c	t1	c	-
5	LOAD R2, d	t1	c	d
6	ADD R1, R1, R2	t1	t2 (=c+d)	d
7	MUL R0, R0, R1	t3 (=t1*t2)	t2	d
8	LOAD R1, e	t3	e	d
9	SUB R0, R0, R1	t4 (final)	e	d
10	STORE R0, result	t4	e	d

Step 5 — Runtime Storage Management

Runtime storage management organizes the memory used during program execution into three areas: the code segment which holds the compiled instructions, the static data segment which holds global and static variables, and the stack segment which holds activation records for each function call.

For the expression $(a - b) * (c + d) - e$, if this expression appears inside a function, all variables a , b , c , d , e and the temporaries $t1$ through $t4$ reside in the function's activation record (stack frame) on the runtime stack.

Step 6 — Stack Frame Layout

High addresses	
+-----+	
Caller saved registers	
Caller local variables	
+-----+	<- Frame Pointer (FP) base
Return address	FP + 0
Saved FP (caller)	FP - 4
Variable a	FP - 8
Variable b	FP - 12
Variable c	FP - 16
Variable d	FP - 20
Variable e	FP - 24
Temp result storage	FP - 28
+-----+	<- Stack Pointer (SP)

Low addresses

FP-relative RISC instructions:

```
LOAD R0, [FP-8]    ; load a
LOAD R1, [FP-12]   ; load b
STORE R0, [FP-28]  ; store final result
```

The frame pointer FP remains fixed throughout the function's execution, providing stable base-relative addresses for all local variables. The stack pointer SP is adjusted at function entry by subtracting the total frame size (28 bytes in this case) and is restored at function exit. This automatic stack management ensures that each function call gets its own private storage area, enabling recursion and concurrent calls without conflicts.

The 10-instruction RISC sequence produced above is optimal for this expression with three registers. No unnecessary spills occur because three registers are sufficient to hold the two operands of each binary operation. If only two registers were available, t1 would need to be spilled after instruction 3 and reloaded before instruction 7, adding two instructions to the sequence.